

UNIVERSIDADE

Curso de Tecnologia em Análise e Desenvolvimento de Sistemas

A3 – ESTUDO DE CASO APLICADO

Google Maps

Otimização de Rotas, Escalabilidade e Algoritmos em Tempo Real

Unidade Curricular: Estruturas de Dados e Análise de Algoritmos

Código: 0006963 | Modalidade: Presencial – Prática (E2A)

Integrantes do Grupo:

Gabriel Alves Bragança - 124221803 | Pedro Picinin Velloso Vieira - 12410337 | Pedro Christovam Marques - 124117694

2025

1. Escolha e Apresentação do Caso

1.1 A Empresa

O Google Maps é um serviço de mapeamento e navegação desenvolvido pela Google LLC, lançado em 2005 e atualmente integrado ao ecossistema do Alphabet Inc. Com mais de 1 bilhão de usuários ativos mensais em mais de 220 países e territórios, o Google Maps é a plataforma de navegação mais utilizada do mundo.

Além da navegação turn-by-turn, o serviço oferece: visualização de mapas em 2D e 3D, Street View, informações de trânsito em tempo real, rotas para pedestres e ciclistas, integração com transporte público, estimativas de tempo de chegada (ETA), informações de estabelecimentos comerciais e avaliações, além de APIs para desenvolvedores.

1.2 Contextualização do Problema

O problema central do Google Maps é o cálculo eficiente de rotas em um grafo gigantesco que representa a malha viária global. Esse grafo contém:

- Mais de 100 milhões de km de estradas mapeadas
- Bilhões de nós (interseções, pontos de referência)
- Trilhões de arestas (segmentos viários com atributos dinâmicos)
- Atualizações em tempo real de tráfego, acidentes, obras e condições climáticas

O desafio não está apenas em encontrar o caminho mais curto entre dois pontos, mas em fazê-lo em milissegundos para milhões de usuários simultâneos, considerando dados dinâmicos que mudam a cada segundo.

1.3 Impacto no Negócio

A qualidade das rotas e a velocidade de resposta têm impacto direto em múltiplas dimensões:

- Experiência do usuário: rotas imprecisas ou lentas afastam usuários para concorrentes (Apple Maps, Waze, HERE)
- Receita publicitária: o Maps é integrado a serviços pagos como Google Ads local, que geram bilhões em receita anual
- APIs para terceiros: empresas como Uber, iFood e Airbnb dependem das APIs do Google Maps
- Credibilidade: um erro de roteamento pode causar acidentes ou atrasos críticos

Uma latência de resposta acima de 500ms já degrada significativamente a experiência do usuário em aplicações móveis, o que torna a otimização algorítmica não apenas técnica, mas estrategicamente vital.

2. Raio-X Técnico do Caso

2.1 Arquitetura e Stack Tecnológica

Componente	Tecnologia / Detalhe
Linguagens	C++ (algoritmos de roteamento críticos), Python (ML e análise de dados), Java/Go (infraestrutura de backend), JavaScript/TypeScript (frontend)
Banco de Dados	Bigtable (dados de mapas NoSQL), Spanner (dados transacionais distribuídos), Datastore (dados de usuário)
Infraestrutura	Google Cloud Platform (GCP), dezenas de data centers distribuídos globalmente, edge nodes para baixa latência
Cache	Memcache distribuído para resultados de rotas frequentes, CDN global para tiles de mapas estáticos
Processamento	MapReduce / Dataflow para pré-computação de grafos, Pub/Sub para eventos de tráfego em tempo real
ML / IA	TensorFlow para modelos de previsão de tráfego, ETAs inteligentes, detecção de incidentes via dados de usuários
APIs	Directions API, Distance Matrix API, Places API — servindo bilhões de requisições/mês
Filas	Google Pub/Sub para ingestão de dados de tráfego, Cloud Tasks para processamento assíncrono

2.2 Problema Computacional Envolvido

O núcleo algorítmico do Google Maps é o Problema do Caminho Mínimo em Grafos Ponderados e Dinâmicos. Entretanto, a versão real do problema vai muito além do clássico Dijkstra:

Por que o problema é difícil?

- Escala absurda: grafos com centenas de bilhões de arestas não cabem na memória de uma única máquina
- Dinamismo: os pesos das arestas (tempo de travessia) mudam em tempo real devido ao tráfego
- Múltiplos critérios: o usuário pode querer a rota mais rápida, mais curta, com pedágio ou sem, evitando rodovias etc. — um problema de otimização multiobjetivo

- Escala de usuários: milhões de consultas simultâneas exigem que o algoritmo seja paralelizável e eficiente em cache
- Consistência eventual: dados de tráfego chegam de fontes heterogêneas (GPS de usuários, sensores, câmeras) com latências diferentes

2.3 Classificação Conceitual do Problema

O problema central de roteamento do Google Maps é conceitualmente classificado como:

- Problema de Caminho Mínimo de Única Fonte (SSSP – Single Source Shortest Path): P, solucionável em tempo polinomial com Dijkstra $O((V + E) \log V)$
- Com restrições adicionais (múltiplos critérios, trânsito dinâmico): aproxima-se de problemas de otimização combinatória, com variantes NP-difíceis
- A versão com K-objetivos simultâneos (tempo, custo, conforto) é classificada como otimização multiobjetivo, geralmente NP-completa na forma exata
- Na prática, o Google adota heurísticas e aproximações para manter a solução em tempo P viável

Problema	Classe	Algoritmo Usado
Caminho mínimo simples	P	Dijkstra / A*
Rota com tráfego em tempo real	Otimização	Contraction Hierarchies + ML
ETA com múltiplos fatores	Otimização / ML	Modelos de aprendizado profundo
Renderização de tiles do mapa	Busca / Indexação	Quadtree + Cache distribuído

2.4 Estratégias Utilizadas pelo Google

a) Contraction Hierarchies (CH)

A principal técnica para roteamento em larga escala é o Contraction Hierarchies, uma otimização sobre o Dijkstra tradicional que funciona em duas fases:

- Pré-processamento (offline): nós de menor importância são "contraídos" (removidos) do grafo, e atalhos são adicionados para preservar os menores caminhos. Isso cria uma hierarquia de vias — estradas expressas têm maior importância que ruas locais.
- Consulta (online): em vez de explorar o grafo inteiro, o algoritmo busca apenas pelas vias de maior hierarquia, reduzindo o espaço de busca em ordens de grandeza.

Com CH, o tempo de consulta em grafos de escala continental cai de minutos (Dijkstra puro) para microssegundos.

b) Algoritmo A* com Heurísticas Geoespaciais

O A* é uma extensão do Dijkstra que usa uma função heurística $h(n)$ para guiar a busca em direção ao destino. No Google Maps, a heurística é a distância Euclidiana (linha reta) ao destino, o que reduz significativamente o número de nós explorados em relação ao Dijkstra puro.

A* garante optimalidade quando a heurística é admissível (nunca superestima o custo real), o que a distância Euclidiana satisfaz em grafos viários.

c) Previsão de Tráfego com Machine Learning

O Google coleta dados anônimos de velocidade e posição de centenas de milhões de dispositivos Android. Esses dados alimentam modelos de ML (redes neurais recorrentes e transformers) que aprendem padrões de tráfego por:

- Hora do dia e dia da semana
- Eventos locais (shows, jogos, feriados)
- Condições climáticas
- Histórico de incidentes

O resultado é um grafo ponderado dinâmico onde os pesos das arestas são previstos com antecedência, não apenas medidos no momento da consulta.

d) Particionamento e Processamento Distribuído

O grafo viário global é particionado geograficamente em tiles (azulejos). Cada tile é processado e servido por diferentes servidores especializados. O balanceamento de carga distribui as requisições para o servidor mais próximo geograficamente do usuário (edge computing), reduzindo latência.

e) Cache Multinível

Rotas populares (ex: aeroporto para centro de uma cidade) são pré-calculadas e armazenadas em cache distribuído (Memcache). Tiles de mapas estáticos são servidos via CDN global. Isso evita recalculiar rotas já conhecidas.

3. Proposta Alternativa do Grupo

A partir da análise do sistema atual, propomos uma abordagem complementar que combina pré-computação inteligente com um grafo temporal adaptativo, aplicando conceitos estudados na disciplina.

3.1 Grafo Temporal com Pesos Probabilísticos

Proposta: substituir os pesos estáticos ou determinísticos das arestas por distribuições de probabilidade baseadas em séries temporais históricas.

Em vez de armazenar apenas o tempo médio de travessia de uma via, cada aresta armazenaria um vetor de médias e desvios padrão indexados por (hora_do_dia, dia_da_semana). Na hora da consulta, o algoritmo escolheria os pesos correspondentes ao horário atual, tornando o grafo inerentemente temporal sem necessidade de recalculá-lo todo o tempo real.

- Estrutura de dados: cada aresta com um array de 168 entradas (24h × 7 dias), cada uma contendo média e desvio padrão do tempo de travessia
- Vantagem: elimina a necessidade de dados de tráfego ao vivo para a maioria das consultas — o histórico já encapsula padrões recorrentes
- Complemento ao CH: os atalhos do Contraction Hierarchies seriam atualizados semanalmente com base nos dados históricos, não em tempo real

3.2 Algoritmo de Roteamento com Tolerância a Incerteza

Proposta: usar programação dinâmica para calcular não apenas o caminho mais rápido esperado, mas também o caminho com menor variância (mais confiável).

Um usuário que precisa chegar a uma reunião às 9h preferiria uma rota que leva 30min em média com desvio de 2min, em vez de uma rota de 25min com desvio de 15min. Isso é um problema de otimização de risco que pode ser modelado como:

$$\text{minimizar: } \alpha \times E[\text{tempo}] + (1-\alpha) \times \text{Var}[\text{tempo}]$$

Onde $\alpha \in [0,1]$ é um parâmetro de preferência do usuário. Com $\alpha=1$, priorizamos velocidade; com $\alpha=0$, priorizamos confiabilidade. Isso poderia ser exposto como uma opção de interface: "Rota mais rápida" vs "Rota mais confiável".

3.3 Indexação por KD-Tree para Geocoding Reverso

Proposta: para a busca de estabelecimentos e pontos de interesse próximos (busca espacial), substituir a busca linear atual por uma KD-Tree (K-Dimensional Tree), uma estrutura de dados especializada em buscas espaciais.

- Complexidade da busca linear: $O(n)$ — examina todos os pontos
- Complexidade com KD-Tree: $O(\sqrt{n})$ em média para buscas por vizinhança
- Para $n = 1$ bilhão de POIs globais, a diferença é de ~ 1 bilhão de operações para ~ 31.623 — uma redução de 99,997%

A KD-Tree seria atualizada incrementalmente (não reconstruída do zero) quando novos POIs são adicionados, mantendo $O(\log n)$ para inserções.

3.4 Heurística Greedy para Múltiplas Paradas

O problema de rota com múltiplas paradas (ex: fazer compras em 5 lojas diferentes) é equivalente ao Problema do Caixeiro Viajante (TSP), que é NP-Completo para a solução ótima.

Proposta: aplicar uma heurística gulosa (Greedy) com melhoramento local (2-opt):

- Fase 1 – Greedy: a partir do ponto de partida, sempre escolher a parada mais próxima ainda não visitada. Complexidade: $O(n^2)$
- Fase 2 – 2-opt: trocar pares de arestas para eliminar cruzamentos de rota. Complexidade: $O(n^2)$ por iteração
- Resultado: solução $\sim 15\text{-}20\%$ acima do ótimo teórico, mas calculada em milissegundos para $n \leq 20$ paradas

Esta abordagem já é utilizada por sistemas similares, mas nossa proposta incluiria a integração com os pesos probabilísticos do item 3.1, tornando o greedy temporalmente consciente.

4. Análise Crítica

4.1 Trade-offs da Solução Atual

Aspecto	Vantagem	Limitação
Contraction Hierarchies	Consultas em microssegundos; ideal para grafos estáticos	Pré-processamento custoso (horas); difícil de atualizar dinamicamente
ML para tráfego	Alta precisão em regiões com muitos usuários	Regiões com poucos usuários têm dados escassos (cold start problem)
Cache de rotas	Reduz drasticamente o custo computacional de rotas populares	Rotas únicas (usuário novo em cidade desconhecida) não se beneficiam do cache

4.2 Solução Ótima vs. Solução Viável

Um insight fundamental da análise é que o Google Maps deliberadamente não busca a solução ótima teórica. O algoritmo de Dijkstra puro encontraria garantidamente o caminho mais curto, mas levaria minutos em um grafo continental. Em vez disso, o sistema busca soluções suficientemente boas em tempo suficientemente curto — um princípio de engenharia chamado *satisficing*.

O Contraction Hierarchies sacrifica flexibilidade (difícil atualização em tempo real) por velocidade de consulta. O A* sacrifica completude (pode não encontrar o ótimo com heurísticas inadmissíveis) por direcionamento eficiente da busca. Isso é aceito porque o usuário não percebe uma diferença de 30 segundos em uma rota de 45 minutos, mas perceberia imediatamente se o aplicativo demorasse 10 segundos para calcular.

4.3 Impacto da Escalabilidade

A escalabilidade é o principal desafio de engenharia do Google Maps. Alguns números ilustram a magnitude:

- 1 bilhão de usuários ativos mensais
- Pico de ~100 milhões de consultas de rota por dia
- Dados de tráfego atualizados a cada 2 minutos para todas as regiões monitoradas
- O grafo viário global ocupa petabytes de armazenamento

Nenhum algoritmo de caminho mínimo clássico, rodando em um único servidor, poderia atender a esse volume. A solução é intrinsecamente distribuída: particionamento do grafo, processamento paralelo, cache em múltiplos níveis e edge computing são todas técnicas arquiteturais que multiplicam a capacidade do sistema.

4.4 Limitações Práticas da Nossa Proposta

Nossa proposta alternativa, embora tecnicamente coerente, enfrenta limitações reais:

- Grafo temporal com 168 entradas por aresta: aumenta o consumo de memória em $\sim 168x$ por aresta. Para um grafo com 10 bilhões de arestas, isso representa terabytes adicionais de dados.
- Otimização de variância: calcular a variância de uma rota inteira requer propagar incertezas ao longo de todas as arestas — matematicamente correto, mas computacionalmente mais custoso.
- KD-Tree para POIs: a árvore precisa ser rebalanceada periodicamente. Em regiões com alta taxa de criação de novos estabelecimentos, isso pode gerar overhead.
- Heurística 2-opt para múltiplas paradas: eficaz para $n \leq 20$ paradas; degrada rapidamente para problemas maiores.

Ainda assim, nossas propostas são complementares e incrementais — não substituem o sistema atual, mas o enriquecem para casos de uso específicos onde o Google Maps ainda tem limitações visíveis para o usuário.

5. Conclusões

O Google Maps é um dos sistemas computacionais mais sofisticados já desenvolvidos, e sua análise revela com clareza os princípios fundamentais da disciplina de Estruturas de Dados e Análise de Algoritmos aplicados em escala real.

A análise demonstra que algoritmos clássicos como Dijkstra e A* são pontos de partida, mas não soluções completas para problemas de escala planetária. O Contraction Hierarchies é um exemplo brilhante de como técnicas de pré-processamento inteligente podem transformar um problema intratável (Dijkstra em $O((V+E) \log V)$ sobre bilhões de nós) em um problema prático resolvido em microssegundos.

Também ficou evidente que a escolha entre solução ótima e solução viável é sempre uma decisão de engenharia com consequências de negócio. O Google Maps prefere consistentemente respostas rápidas e boas o suficiente a respostas perfeitas e lentas — uma filosofia válida e eficaz em sistemas de alta escala.

Finalmente, nosso grupo concluiu que nenhuma solução técnica existe no vácuo: as escolhas algorítmicas do Google Maps são moldadas por restrições de hardware, expectativas de usuário, modelo de negócio e capacidade de manutenção. Isso é engenharia de software real — e é por isso que ela é desafiadora e fascinante.

Referências

GEISBERGER, R. et al. Contraction Hierarchies: Faster and Simpler Hierarchical Routing in Road Networks. In: Experimental Algorithms, 2008. Springer, Berlin, Heidelberg.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, v. 4, n. 2, p. 100-107, 1968.

GOOGLE. Google Maps Platform Documentation. Disponível em: <https://developers.google.com/maps/documentation>. Acesso em: jun. 2025.

GOOGLE AI BLOG. Traffic prediction with advanced Graph Neural Networks. 2020. Disponível em: <https://ai.googleblog.com/2020/09/traffic-prediction-with-advanced-graph.html>

DIJKSTRA, E. W. A note on two problems in connexion with graphs. Numerische Mathematik, v. 1, p. 269-271, 1959.

CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. MIT Press, 2022.

GOOGLE ENGINEERING. Under the hood of Google Maps routing. Google Tech Talks, 2019. Disponível em: <https://www.youtube.com/watch?v=BteXSqxS6L8>

SKIENA, S. The Algorithm Design Manual. 3. ed. Springer, 2020.

ZHANG, H. et al. Deep learning for real-time ETA estimation in Google Maps. Google Research, 2022. Disponível em: <https://research.google/pubs/>

AWASTHI, P. et al. Routing Millions of Vehicles in Real Time with Machine Learning. Google AI Blog, 2023. Disponível em: <https://ai.googleblog.com/>